

<<Auto Robot>>

<<COARITE COARITE, IVAN ENRIQUE; GALVEZ, EZEQUIEL
MAXIMILIANO; GRAMAJO HECTOR MARCELO; NOVOA ROMERO,
MARTÍN RICARDO; RIOS DI GAETA, BAUTISTA>>
<<94447552; 37659307; 34519918; 43087221; 46431397>>
<<Martes, GRUPO M4>>

¹ Universidad Nacional de La Matanza,
Departamento de Ingenieria e Investigaciones Tecnologicas,
Florencio Varela 1903 - San Justo, Argentina

Resumen.

Auto Robot es un sistema IoT compuesto por un ESP32 y una aplicacion Android. El sistema permite controlar un vehiculo robotico de forma remota, detectar obstaculos, medir iluminacion ambiente y visualizar telemetria desde un smartphone. El ESP32 utiliza sensores ultrasonicos HC-SR04 para detectar obstaculos, un sensor LDR para medir luz, motores DC con puente H L298N para el movimiento y un LED como actuador de baja iluminacion. La logica embebida se organiza con FreeRTOS, colas de eventos y una maquina de estados finita. La aplicacion Android se comunica con el embebido mediante MQTT, permite enviar comandos manuales de movimiento y tambien controlar el robot con el sensor de rotacion del telefono. Para la actividad de IA de la comision martes se comparan metricas de CPU y memoria de la app Android manual y la app Android con vibecoding usando el script ADB provisto por la catedra.

Palabras claves: ESP32, FreeRTOS, MQTT, Android, IoT, Sensores.

1. Introduccion

Auto Robot presenta una solucion tecnologica para controlar y monitorear un vehiculo robotico integrando hardware embebido, comunicacion IoT y software movil. La utilidad principal de la aplicacion es permitir que el usuario controle el movimiento del robot desde Android y observe en tiempo real los valores publicados por el ESP32.

El sistema funciona de la siguiente manera:

- El ESP32 actua como controlador principal del auto. Lee dos sensores ultrasonicos, un sensor LDR y ejecuta una maquina de estados para decidir el movimiento.
- Si detecta un obstaculo del lado izquierdo mientras avanza, genera una maniobra de evasion hacia la derecha.
- Si detecta un obstaculo del lado derecho mientras avanza, genera una maniobra de evasion hacia la izquierda.
- Si el camino vuelve a estar libre, el robot puede retomar el avance.
- Si detecta baja luz mediante el LDR, enciende el LED; cuando vuelve la luz, lo apaga.
- La aplicacion Android se conecta al mismo broker MQTT que el ESP32 y permite dos funciones principales.
- Monitoreo y control: el usuario envia comandos forward, backward, left, right y stop, y visualiza distancia izquierda, distancia derecha, luz y estado actual.
- Control por sensor: el usuario inclina el telefono y la app usa TYPE_ROTATION_VECTOR para traducir la orientacion del smartphone en comandos de movimiento.

2. Desarrollo

2.1 Repositorios

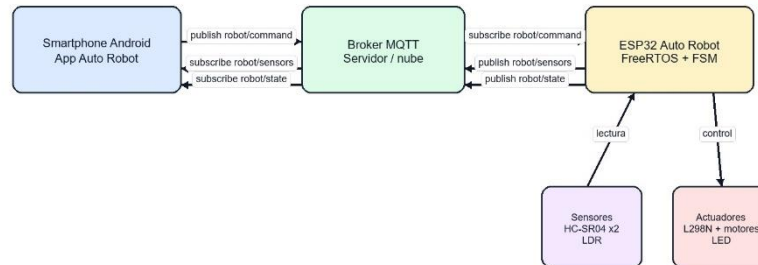
GitHub: <https://github.com/UNLAM-SOA/2026-SOA-Q1-M4>

Wokwi: <https://wokwi.com/projects/463207907056147457>

2.2 Diagrama Funcional

Diagrama funcional del sistema:

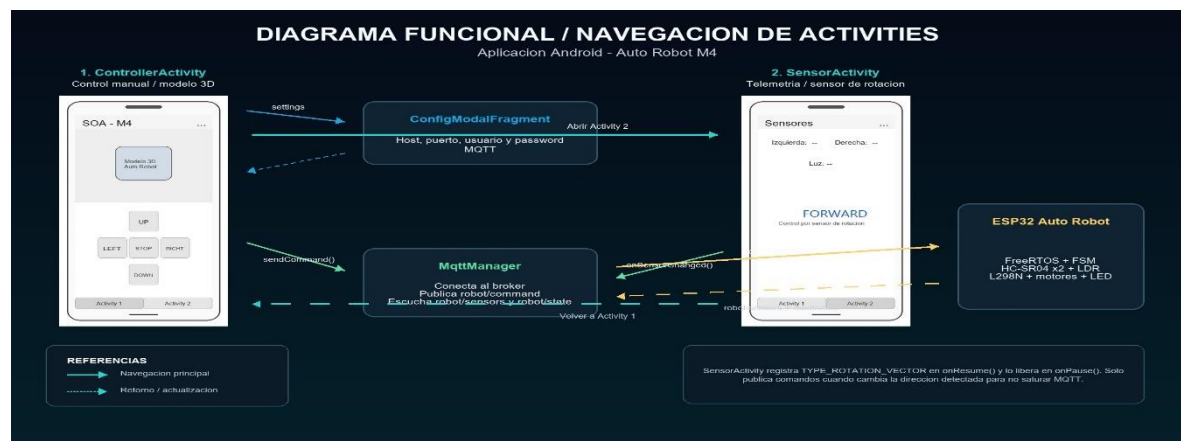
Arquitectura IoT - Auto Robot ESP32 + Android



Flujo principal: la app envía comandos por MQTT, el ESP32 los transforma en eventos de la máquina de estados, controla motores/LED y publica telemetría.

Formato sensores esperado por la APK: distanciaIzquierda;distanciaDerecha;luz. Ejemplo: 12;18;640

Diagrama de Activities de Android:



Flujo de datos (MQTT)

ESP32 -> Broker -> App (Publica en):

robot/sensors: distancia izquierda, distancia derecha y nivel de luz con formato distanciaIzquierda;distanciaDerecha;luz.

robot/state: estado actual de la máquina de estados del ESP32.

App -> Broker -> ESP32 (Publica en):

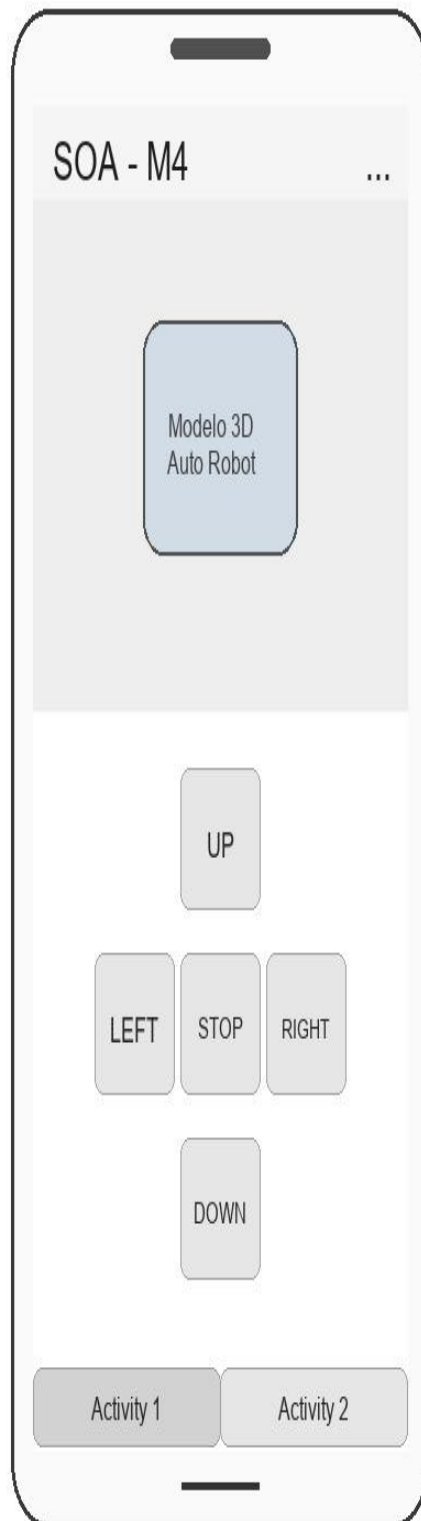
robot/command: comandos de movimiento de la app (forward, backward, left, right, stop).

El broker MQTT desacopla la aplicación Android del ESP32. La app no se comunica directamente con el microcontrolador, sino que publica y consume mensajes mediante topics.

2.3 Manual de Usuario (App Android e Interacción)

ControllerActivity

Es la pantalla principal de control manual y visualizacion del auto.



Views y controles principales:

- Modelo 3D del auto: muestra una representacion visual del estado de movimiento.
- Boton arriba: envia el comando forward al topic robot/command.
- Boton abajo: envia el comando backward al topic robot/command.
- Boton izquierda: envia el comando left al topic robot/command.
- Boton derecha: envia el comando right al topic robot/command.
- Boton stop: envia el comando stop al topic robot/command.
- Boton sensores: inicia SensorActivity.
- Menu configuracion: abre ConfigModalFragment para configurar host, puerto, usuario y password del broker MQTT.

SensorActivity

Esta pantalla permite ver telemetria del ESP32 y controlar el robot inclinando el telefono.



Views y comportamiento:

- leftDistanceText: muestra la distancia izquierda recibida desde robot/sensors.
- rightDistanceText: muestra la distancia derecha recibida desde robot/sensors.
- lightText: muestra el valor de luz recibido desde robot/sensors.
- txtSensorOutput: muestra el comando calculado por el sensor del telefono.
- Boton controller: vuelve a la pantalla principal.

Control por sensor Android:

- La app usa SensorManager.
- Se registra como SensorEventListener.
- Usa el sensor TYPE_ROTATION_VECTOR.
- En onSensorChanged() calcula pitch y roll.
- Si el telefono supera el umbral configurado, traduce la inclinacion en forward, backward, left o right.
- Si vuelve a posicion neutra, envia stop.
- Solo publica un comando si la direccion cambio, para no saturar el broker MQTT.

Interaccion con el ESP32 (FSM):

El ESP32 recibe los comandos publicados por Android en robot/command y los transforma en eventos para su maquina de estados:

forward -> MOVE_FORWARD

backward -> MOVE_BACKWARD

left -> MOVE_LEFT

right -> MOVE_RIGHT

stop -> STOP

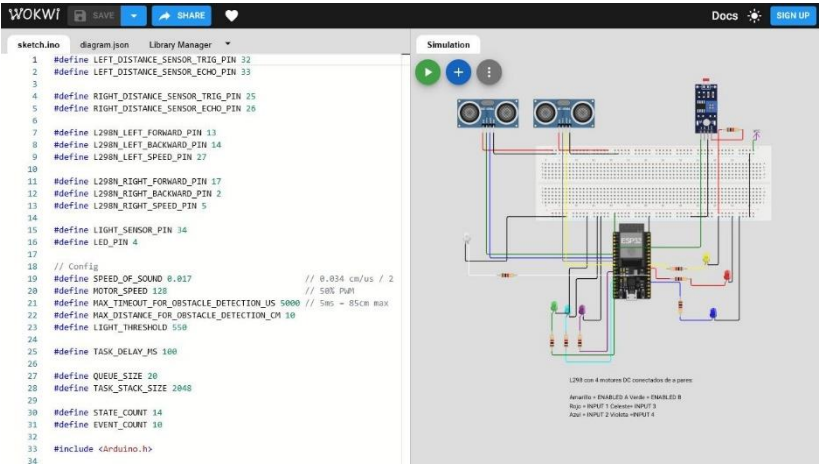
La FSM del ESP32 decide la accion final. Por ejemplo, si el usuario solicita avanzar pero se detecta un obstaculo izquierdo, el sistema prioriza la evasion correspondiente.

2.4 Circuito del ESP32 (Wokwi)

El circuito del ESP32 utiliza:

- ESP32.
- Dos sensores ultrasonicos HC-SR04.
- Sensor LDR.
- LED indicador de baja luz.
- Puente H L298N.
- Motores DC.

Proyecto Wokwi: <https://wokwi.com/projects/463207907056147457>



2.5 Especificacion de Componentes

Componente	Tipo	Funcion en el sistema	Senal
ESP32	Unidad de control	Ejecuta FreeRTOS, procesa la FSM, conecta WiFi/MQTT y controla sensores/actuadores.	Digital / ADC / PWM
HC-SR04 izquierdo	Sensor	Mide distancia del lado izquierdo para detectar obstaculos.	Digital: trigger/echo
HC-SR04 derecho	Sensor	Mide distancia del lado derecho para detectar obstaculos.	Digital: trigger/echo
LDR	Sensor	Mide nivel de luz ambiente.	Analoga ADC
LED	Actuador	Indica baja iluminacion.	Digital
L298N	Driver de actuadores	Permite controlar sentido y velocidad de los motores DC.	Digital + PWM
Motores DC	Actuadores	Ejecutan avance, retroceso, giro y detencion del auto.	PWM / potencia
Smartphone Android	Interfaz IoT	Envia comandos, muestra telemetria y permite control por sensor de rotacion.	MQTT / sensor Android
Broker MQTT	Intermediario IoT	Desacopla la app Android del ESP32 mediante topics.	TCP/IP

2.6 Diagrama de Estados

Diagrama de estados resumido del embebido:

MAQUINA DE ESTADOS RESUMIDA - AUTO ROBOT ESP32

Resumen funcional: comandos, obstaculos, STOP y luz

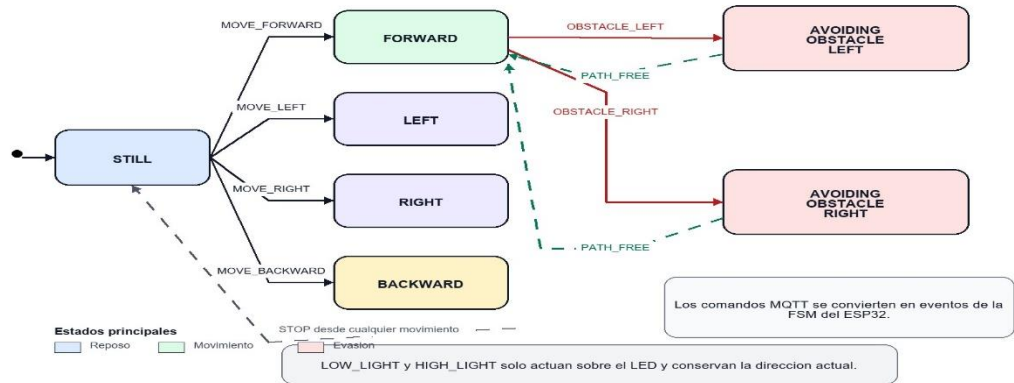
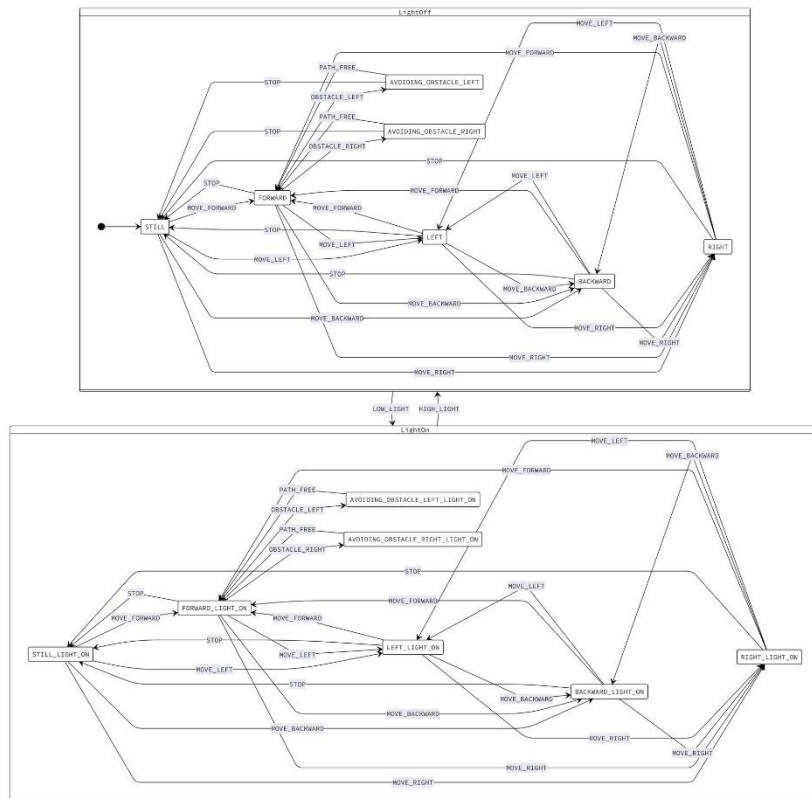


Diagrama detallado generado a partir de la implementación:



Estados principales:

- STILL
- FORWARD
- BACKWARD
- LEFT

- RIGHT
- AVOIDING_OBSTACLE_LEFT
- AVOIDING_OBSTACLE_RIGHT

En la implementacion manual tambien existen variantes *_LIGHT_ON. Estas variantes representan el mismo movimiento, pero con el LED encendido por baja iluminacion.

Eventos principales:

- MOVE_FORWARD
- MOVE_BACKWARD
- MOVE_LEFT
- MOVE_RIGHT
- STOP
- OBSTACLE_LEFT
- OBSTACLE_RIGHT
- PATH_FREE
- LOW_LIGHT
- HIGH_LIGHT

Transiciones principales:

Estado actual	Evento	Proximo estado	Accion
STILL	MOVE_FORWARD	FORWARD	Avanzar
STILL	MOVE_BACKWARD	BACKWARD	Retroceder
STILL	MOVE_LEFT	LEFT	Girar a izquierda
STILL	MOVE_RIGHT	RIGHT	Girar a derecha
Cualquier movimiento	STOP	STILL	Detener motores
FORWARD	OBSTACLE_LEFT	AVOIDING_OBSTACLE_LEFT	Evadir hacia la derecha
FORWARD	OBSTACLE_RIGHT	AVOIDING_OBSTACLE_RIGHT	Evadir hacia la izquierda
AVOIDING_OBSTACLE_LEFT	PATH_FREE	FORWARD	Retomar avance
AVOIDING_OBSTACLE_RIGHT	PATH_FREE	FORWARD	Retomar avance

Cualquier estado	LOW_LIGHT	Estado equivalente con LED encendido	Encender LED
Cualquier estado con LED	HIGH_LIGHT	Estado equivalente con LED apagado	Apagar LED

2.7 Metricas Android

Para la comision martes, las metricas de CPU y memoria se realizan sobre la aplicacion Android usando el script `get_cpu_mem_smarthpone.sh` de la catedra.

Versiones medidas:

- Android manual: dev.mnovoa.SOA.
- Android vibecoding: dev.mnovoa.SOA.vibecoding.

Casos de prueba:

- Manual en reposo durante 10 segundos.
- Manual con accion de sensor Android durante 10 segundos.
- Vibecoding en reposo durante 10 segundos.
- Vibecoding con accion de sensor Android durante 10 segundos.

Los resultados se documentan en `Informes/metricasCPU.md` y en el informe de IA.

3. Conclusiones

3.1 Recaudos

El diseno del sistema se centro en separar responsabilidades entre Android, broker MQTT y ESP32.

ESP32 (FreeRTOS): la maquina de estados concentra la logica de transicion. Las tareas de lectura y comunicacion generan eventos que luego son procesados por la FSM.

Actuadores no bloqueantes: el control de motores y LED se mantiene separado de la lectura de sensores para evitar que una accion fisica detenga el resto del sistema.

Concurrencia MQTT: el uso de topics definidos permite ordenar el flujo de datos entre app y embebido, evitando mezclar comandos, sensores y estado.

Android (UI Thread): las operaciones de sensores y comunicacion MQTT no deben bloquear el hilo principal. La UI solo refleja el estado y permite enviar comandos.

Android (Ciclo de Vida): el sensor del telefono se registra en `onResume()` y se libera en `onPause()` para evitar consumo innecesario de bateria.

3.2 Problemas Encontrados y Resueltos

Problema 1

El contrato de comunicacion entre Android y ESP32 necesitaba quedar unificado.

Causa: si la app y el embebido usan topics o formatos distintos, el sistema no puede intercambiar datos correctamente.

Solucion: se definieron tres topics principales: `robot/command`, `robot/sensors` y `robot/state`. Ademas, `robot/sensors` usa el formato `distanciaIzquierda;distanciaDerecha;luz`.

Problema 2

El control por sensor podia publicar demasiados comandos MQTT.

Causa: `onSensorChanged()` se ejecuta muchas veces por segundo.

Solucion: la app guarda la ultima direccion enviada y solo publica un nuevo comando cuando cambia la direccion detectada.

Problema 3

Conflicto de pines en la placa ESP32.

Causa: durante el armado se detecto que un pin de la placa, identificado en el circuito como GPIO 32, quedaba usado por una conexion y al mismo tiempo se lo estaba considerando para otra parte del sistema. Esto generaba conflicto en el cableado y hacia que una de las lecturas no funcionara como se esperaba.

Solucion: se reviso el pinout del ESP32, se separaron las conexiones para que cada sensor y actuador use un GPIO libre, y se actualizo el cableado/defines del proyecto para evitar compartir el mismo pin.

Problema 4

La consigna de metricas cambio de ESP32 a Android.

Causa: inicialmente se habia preparado la medicion con `Metrics.h`, pero la clase de IA para la comision martes pide medir Android con el script ADB.

Solucion: se adapto el informe de IA y `metricasCPU` al script `get_cpu_mem_smarthpone.sh`. Tambien se separo el `applicationId` de la version vibecoding para poder instalar y medir ambas APK en el mismo telefono.

3.3 Lecciones Aprendidas

- La arquitectura IoT con MQTT permite desacoplar el ESP32 de la aplicación Android.
- FreeRTOS facilita separar lectura de sensores, procesamiento de eventos y acciones del robot.
- La máquina de estados finita permite organizar el comportamiento del embebido de forma clara y mantenible.
- En Android es importante respetar el ciclo de vida de Activities y sensores para evitar consumo innecesario.
- Las métricas deben medirse en el dispositivo real, porque CPU y memoria dependen del teléfono, la versión de Android y los procesos activos.
- Separar versión manual y versión vibecoding permite comparar funcionalidad, consumo y tiempo de desarrollo de manera ordenada.

4. Referencias

1. Catedra de Sistemas Operativos Avanzados, "Material de clases", Universidad Nacional de La Matanza. Disponible: https://www.soa-unlam.com.ar/wiki/index.php/PUBLICO:Material_de_clases
2. Catedra de Sistemas Operativos Avanzados, "Uso de CPU y memoria en Android", repositorio Material-SOA. Disponible: <https://gitlab.com/so-unlam/Material-SOA/-/tree/master/Ejemplos%20Android/UseCpuMemAndroid>
3. Espressif Systems, "ESP32 Arduino Core Documentation". Disponible: <https://docs.espressif.com/projects/arduino-esp32/>
4. OASIS, "MQTT Version 3.1.1". Disponible: <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/>